Brandon Lee, Edens Guerrier, Tevfik John

Professor Gu

CSC112

October 30th, 2026

Lab 1 Report: Printing Vowels and Consonants

Introduction

All three programs compile and produce the required output. In each, one worker prints words starting with a vowel and the other prints words starting with a consonant. The tasks differ in how the two workers are coordinated. Task 1 uses a mutex to keep two concurrent threads in input order; Tasks 2 and 3 use pthread_join() and waitpid() to run the workers one after the other. Tasks 1 and 2 ran on Windows with GCC 15.2.0 (MSYS2). Task 3 ran in Ubuntu on WSL, because fork() and sys/wait.h do not exist on Windows.

Task 1

Both threads run at once and share current_index, the position of the next word. Each loop iteration locks the mutex, exits if every word is done, prints the current word only if it belongs to this thread, then unlocks:

```c
void *print_vowels(void *arg) {
    while (1) {
        pthread_mutex_lock(&mutex);
        if (current_index >= word_count) {
            pthread_mutex_unlock(&mutex);
            break;
        }
        if (is_vowel(words[current_index][0])) {
            printf("Vowel: %s\n", words[current_index]);
            current_index++;
        }
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}
```

print_consonants() is the same with !is_vowel(). Order is preserved because current_index only advances under the lock, and only by the thread that owns the current word.

Our first version kept the starter code's turn variable and flipped it after every print. It hung on the lab's example: after `grape`, the turn went to the vowel thread, but `kiwi` is a consonant, so neither thread could continue. It also checked current_index outside the lock, which could let a thread read past the last word. Removing turn and moving the check inside the lock fixed both problems.

All five test inputs printed in input order, and none hit the 3-second [HUNG] timeout, including consecutive consonants and a consonant first.

```
PS C:\Users\brand> gcc -pthread -o task1 task1.c
PS C:\Users\brand> $tests = @(
>>   @("apple","banana","orange","grape","kiwi","umbrella"),
>>   @("banana","apple"),
>>   @("banana"),
>>   @("apple","kiwi","grape"),
>>   @("Apple","Banana","Egg","Tree")
>> )
PS C:\Users\brand> foreach ($t in $tests) {
>>   "--- $($t -join ' ')"
>>   $p = Start-Process .\task1.exe -ArgumentList $t -NoNewWindow -PassThru
>>   if (-not $p.WaitForExit(3000)) { $p.Kill(); "[HUNG]" }
>> }
--- apple banana orange grape kiwi umbrella
Vowel: apple
Consonant: banana
Vowel: orange
Consonant: grape
Consonant: kiwi
Vowel: umbrella
--- banana apple
Consonant: banana
Vowel: apple
--- banana
Consonant: banana
--- apple kiwi grape
Vowel: apple
Consonant: kiwi
Consonant: grape
--- Apple Banana Egg Tree
Vowel: Apple
Consonant: Banana
Vowel: Egg
Consonant: Tree
PS C:\Users\brand>
```

Task 2

Each thread function loops over the whole word list and prints only its own type. The ordering comes from main(), which waits for the vowel thread to finish before creating the consonant thread:

```
pthread_create(&vow_thread, NULL, print_vowels, NULL);
pthread_join(vow_thread, NULL);       // wait until every vowel word
is printed
pthread_create(&cons_thread, NULL, print_consonants, NULL);
pthread_join(cons_thread, NULL);
```

The threads never overlap, so no mutex is needed. The mutex, turn, current_index and unused includes from Task 1 were removed.

In all eight test inputs, every Vowel line appears before the first Consonant line, with each group in input order. The inputs include consonants only, a single word, mixed capitals, repeated words, and words starting with y, which count as consonants.

```
PS C:\Users\brand> $tests = @(
>>   @("apple","banana","orange","grape","kiwi","umbrella"),
>>   @("tree","dog","kiwi"),
>>   @("apple"),
>>   @("banana"),
>>   @("Apple","Banana","Egg","Tree"),
>>   @("banana","apple"),
>>   @("kiwi","apple","kiwi","apple"),
>>   @("yellow","zoo")
>> )
PS C:\Users\brand> foreach ($t in $tests) {
>>   "--- $($t -join ' ')"
>>   .\task2.exe @t
>> }
--- apple banana orange grape kiwi umbrella
Vowel: apple
Vowel: orange
Vowel: umbrella
Consonant: banana
Consonant: grape
Consonant: kiwi
--- tree dog kiwi
Consonant: tree
Consonant: dog
Consonant: kiwi
--- apple
Vowel: apple
--- banana
Consonant: banana
--- Apple Banana Egg Tree
Vowel: Apple
Vowel: Egg
Consonant: Banana
Consonant: Tree
--- banana apple
Vowel: apple
Consonant: banana
--- kiwi apple kiwi apple
Vowel: apple
Vowel: apple
Consonant: kiwi
Consonant: kiwi
--- yellow zoo
Consonant: yellow
Consonant: zoo
PS C:\Users\brand>
```

Task 3

The print functions are the same as in Task 2, but each runs in a child process created with fork(). fork() returns 0 in the child and the child's PID in the parent, so each process checks the return value:

```c
pid_t vow_pid = fork();
if (vow_pid < 0) {
    perror("fork");
    return EXIT_FAILURE;
}
if (vow_pid == 0) {
    print_vowels(NULL);       // child: prints, then calls
}
waitpid(vow_pid, NULL, 0);    // parent waits for the vowel child

pid_t cons_pid = fork();
if (cons_pid < 0) {
    perror("fork");
    return EXIT_FAILURE;
}
if (cons_pid == 0) {
    print_consonants(NULL);
}
waitpid(cons_pid, NULL, 0);
```

Each child ends with exit(0); without it, the vowel child would continue into main, run the second fork() itself, and print the consonants twice. waitpid() between the forks guarantees the order. No mutex is needed, because each child has its own copy of memory and the children never run at the same time.

Built in WSL with `gcc -o task3 task3.c`. All seven inputs printed vowels before consonants. The commands were pasted together, so the outputs follow in command order: the 6-word example, `apple egg orange`, `tree dog kiwi`, `apple`, `banana`, `Apple Banana Egg Tree`, and `banana apple`.

```
brandonjlee1214@TABLET-3D1CHAVM:/mnt/c/Users/brand$ ./task3 apple banana orange grape kiwi umbrella
./task3 apple egg orange
./task3 tree dog kiwi
./task3 apple
./task3 banana
./task3 Apple Banana Egg Tree
./task3 banana apple
Vowel: apple
Vowel: orange
Vowel: umbrella
Consonant: banana
Consonant: grape
Consonant: kiwi
Vowel: apple
Vowel: egg
Vowel: orange
Consonant: tree
Consonant: dog
Consonant: kiwi
Vowel: apple
Consonant: banana
Vowel: Apple
Vowel: Egg
Consonant: Banana
Consonant: Tree
Vowel: apple
Consonant: banana
```

Alternatives Tried

Each alternative only reorders the create/join or fork/wait calls in main(). Task 1 hangs; Tasks 2 and 3 usually still print correctly but lose their guarantee.

The Task 1 alternative joins the vowel thread before creating the consonant thread. With this change, the program hangs on any input that contains a consonant word. For example, `apple egg orange` finishes, but `apple banana orange` never does. The vowel thread cannot print `banana`, so it keeps spinning while it waits for current_index to move. Only the consonant thread can move it, and that thread is never created, because main is blocked in pthread_join(). This shows that Task 1's design needs both threads running at the same time.

```
PS C:\Users\brand> gcc -pthread -o task1_alt task1_alt.c
PS C:\Users\brand> .\task1_alt.exe apple egg orange
Vowel: apple
Vowel: egg
Vowel: orange
PS C:\Users\brand> .\task1_alt.exe apple banana orange
Vowel: apple
```

The Task 2 alternative creates both threads before joining either one. The output is usually still correct, but the order is no longer guaranteed. Both threads run concurrently, so the scheduler decides which one prints when. After many runs with long word lists, the order was always correct, because each thread finishes in microseconds. This is luck rather than design, and on a busy machine the lines could interleave.

The alternative forks both children before waiting for either one. This creates the same race as in Task 2, but with processes instead of threads. After many runs with long word lists, a run occasionally printed out of order. In addition, the lab's snippet has no if (pid == 0) check between the two forks, so the vowel child also runs the second fork() and three children are created instead of two.

Conclusion

A mutex protects shared data while workers run at the same time, and joining or waiting controls which worker runs first. Task 1 needed the mutex because both threads changed current_index concurrently, while Tasks 2 and 3 needed no lock because pthread_join() and waitpid() made the workers run one after the other. Threads share memory, whereas forked children each get their own copy and require a POSIX system such as Linux. Finally, the Task 1 hang showed that code that works on easy inputs can still fail, and testing edge cases is what exposed it.